# TP9 - Unidad 7: Food Store en MongoDB

Santiago Barretto | DB_Santiago_Barretto | rama codex/tp9-unidad7-nosql | 02/11/2026

## 1. Modelo documental y migración

Desde PostgreSQL 18.4 se exportaron 20 pedidos existentes y 40 líneas de la carga sintética Food Store. MongoDB 9.0.4 contiene una colección foodstore_tp9.pedidos con 20 documentos. Se conservaron los IDs originales, la forma de pago, las cantidades y el precio histórico de cada línea. El total se calculó con esos precios, no con el precio de lista actual.

| PostgreSQL | Documento MongoDB |
|---|---|
| pedido (PK id_pedido, fecha, forma_pago) | _id/id_pedido Long; fecha Date + fecha_pg_exacta; forma_pago |
| cliente (FK id_cliente) | cliente embebido: id_cliente, nombre y apellido sintéticos |
| detalle_pedido (FK pedido/producto) | lineas[]: id_producto, cantidad, precio_unitario y subtotal |
| SUM(cantidad * precio_unitario) | total Decimal128; precios y subtotales también Decimal128 |

El origen no tiene columna estado. Se usó la marca explícita sin_estado_en_origen en los documentos; no representa un estado comercial. Se omitieron email y teléfono. El texto fecha_pg_exacta preserva los microsegundos que BSON Date no retiene.

### Documento real de ejemplo: pedido 400001

| Producto | Cantidad | Precio histórico | Subtotal |
|---|---|---|---|
| 100001 | 2 | 537.00 | 1074.00 |
| 100024 | 4 | 1388.00 | 5552.00 |

Cliente ID 40001; forma de pago TARJETA; total 6626.00. La colección tiene índices por ID, estado, producto embebido y total.

### Reglas preservadas

(1) Cada pedido tiene al menos una línea, cantidad positiva y precio no negativo; subtotal y total coinciden con la aritmética exacta en centavos. (2) Los IDs de pedido son únicos y las referencias de cliente/producto proceden de FK válidas en PostgreSQL. La verificación posterior comparó todos los campos y MongoDB rechazó una inserción con cantidad cero.

## 2. Consultas y verificación entre motores

Las consultas se ejecutaron sobre la misma muestra de 20 pedidos. MongoDB mostró todos los IDs y su marca de estado (consulta A). Para el producto se eligió el ID 100001, presente en ocho pedidos. El umbral 10.000,00 separa seis pedidos; los totales van de 3.418,00 a 26.099,00.

| Consulta | PostgreSQL | MongoDB | IDs |
|---|---|---|---|
| Estado ausente | 20 | 20 | Coinciden |
| Producto 100001 | 8 | 8 | Coinciden |
| Total > 10.000 | 6 | 6 | Coinciden |
| Pago TARJETA (extra) | 7 | 7 | Coinciden |

IDs por estado (B): 400001, 400002, 400003, 400004, 400005, 400006, 400007, 400008, 400009, 400010, 400011, 400012, 400013, 405882, 450001, 455882, 500001, 505882, 550001, 555882

IDs por producto (C): 400001, 405882, 450001, 455882, 500001, 505882, 550001, 555882

El filtro de estado devuelve los 20 por ausencia del dato en el origen. Como contraste selectivo con un atributo real se añadió forma_pago = TARJETA: 7 IDs idénticos en ambos motores.

## 3. Dos afirmaciones de IA contrastadas

Lectura del pedido completo. OpenCode indicó que embebiendo líneas se evita el JOIN. La documentación oficial de MongoDB permite recuperar datos relacionados en una operación, y la consulta real devuelve cabecera y líneas. Esto no demuestra que sea más rápido: no se midió latencia.

Coherencia y atomicidad. La afirmación de que MongoDB deja toda la coherencia a la aplicación era demasiado amplia. Sus escrituras son atómicas por documento y admite transacciones multidocumento en despliegues compatibles. Este laboratorio embebe cabecera y líneas y valida cantidades; la igualdad de totales se comprobó en el script. PostgreSQL conserva las claves foráneas observadas entre tablas.

## 4. Conclusión

Conviene mantener PostgreSQL como fuente de verdad de Food Store y considerar MongoDB sólo como copia de lectura para pedidos completos si una medición posterior justifica el costo de sincronizarla. El documento embebido simplifica esa lectura y permite actualizar cabecera y líneas de forma atómica en un solo documento. En cambio, PostgreSQL ya protege las relaciones entre cliente, pedido, detalle y producto mediante claves foráneas, y conserva los precios históricos con NUMERIC. La migración preservó IDs, cantidades y precios mediante Decimal128, y los 20 documentos comparados devolvieron exactamente los mismos IDs en ambos motores. No medimos latencia ni concurrencia, por lo que no afirmamos que MongoDB sea más rápido. Además, pedido no guarda estado: la marca sin_estado_en_origen evita atribuirle un dato inexistente y señala una limitación funcional que debería resolver el modelo de negocio antes de usar estados reales.

Evidencia íntegra y scripts: tp9/evidencia/20261102_133900/, consultas_mongodb.js y consultas_postgresql.sql. No se incluyeron datos personales reales ni mediciones de rendimiento inventadas.